AWS Level 3 iteration 3 terminal documentation

Claudio Verdi | APL | 24 October 2026

I created an Ubuntu instance with Terraform. User-data installed and configured Nginx, then fetched and deployed the website automatically at first boot. The cloud-init log, HTTP test, content test and browser confirmed the result. I then removed the instance and verified its terminated state.

Environment and resources

Property	Value
Region	eu-north-1
Instance ID	i-0f2de6eed9ef7bf43
Instance type	t3.micro
AMI	ami-06fb1802e2b757f09
Subnet	subnet-05d4dca4820d2eb94
Security group	sg-0fb87d0f08a213466
Key pair	devops.school.level3.tabish-admin
Former public IP	13.62.54.224
Private IP	172.31.18.4
AWS provider	hashicorp/aws 6.70.0
Nginx / Git	1.24.0 / 2.43.0
Root disk	8 GiB gp3 / encrypted / delete_on_termination

Reference: Level3 Iteration3(3).sh. I adapted the commands to Ubuntu and the website's actual file structure.

Preparation and automation

I created ~/aws-devops-level3-iteration3 and copied five configuration files from iteration 2 without Terraform state. I reused the region, AMI, subnet, security group and key pair.

The user-data.web.sh script uses apt-get, clones into /opt/hello-web and deploys heyme as /var/www/html/index.html with mode 644. bash -n checked its syntax locally.

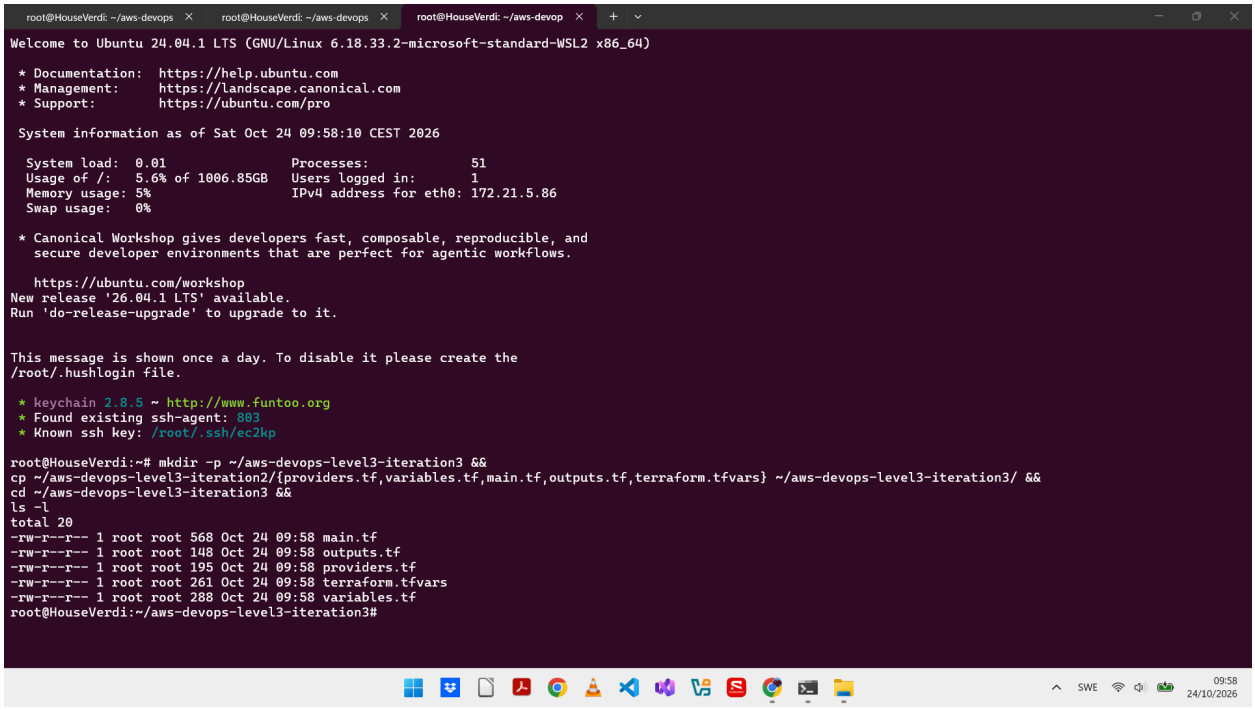


Figure 1. Five configuration files in the new directory

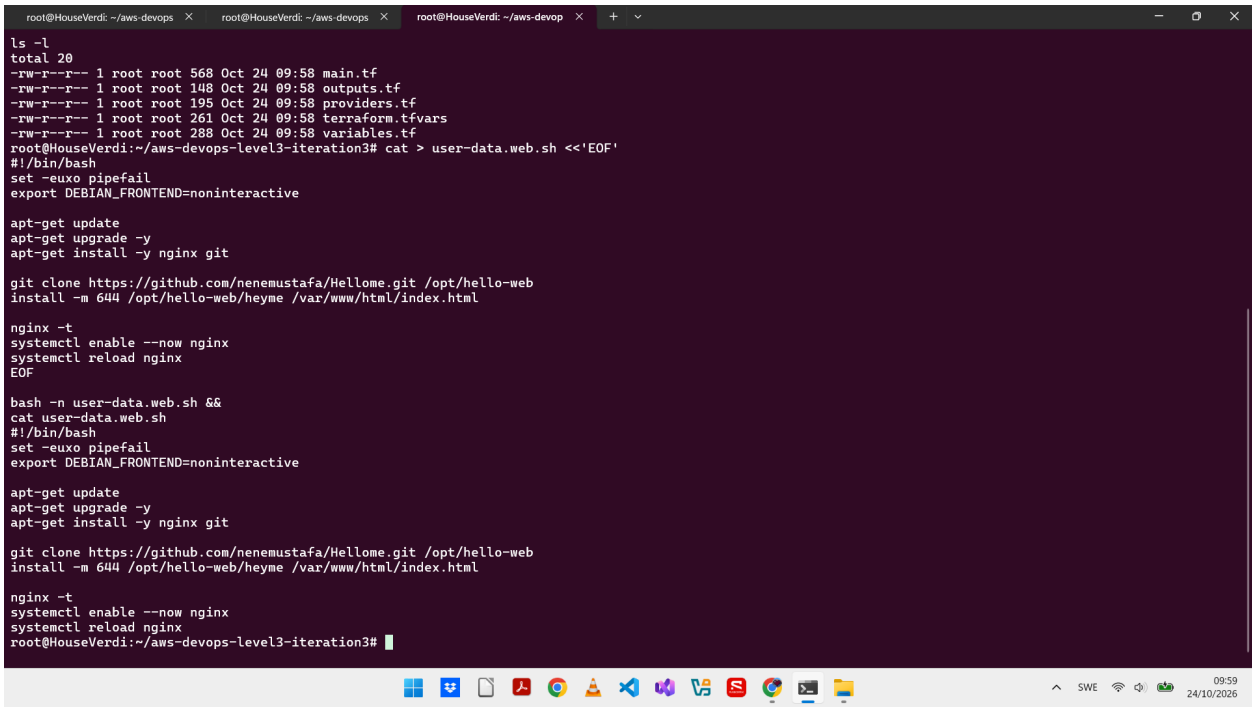


Figure 2. User-data with automatic deployment and successful syntax check

Terraform validation and planning

main.tf reads user-data.web.sh using file() and sets user_data_replace_on_change=true. IMDSv2 is required and the root disk is encrypted. terraform fmt, init and validate completed. The plan was saved as tfplan and showed 1 to add, 0 to change and 0 to destroy.

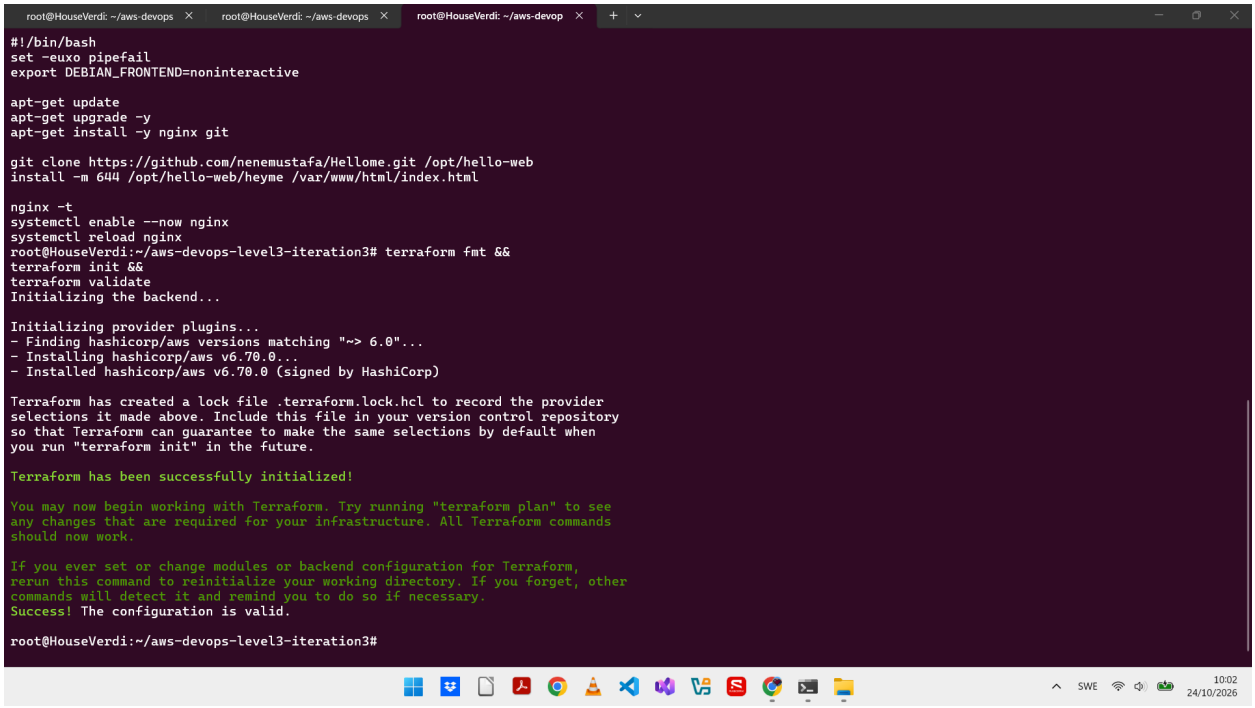


Figure 3. Initialization with provider 6.70.0 and valid configuration

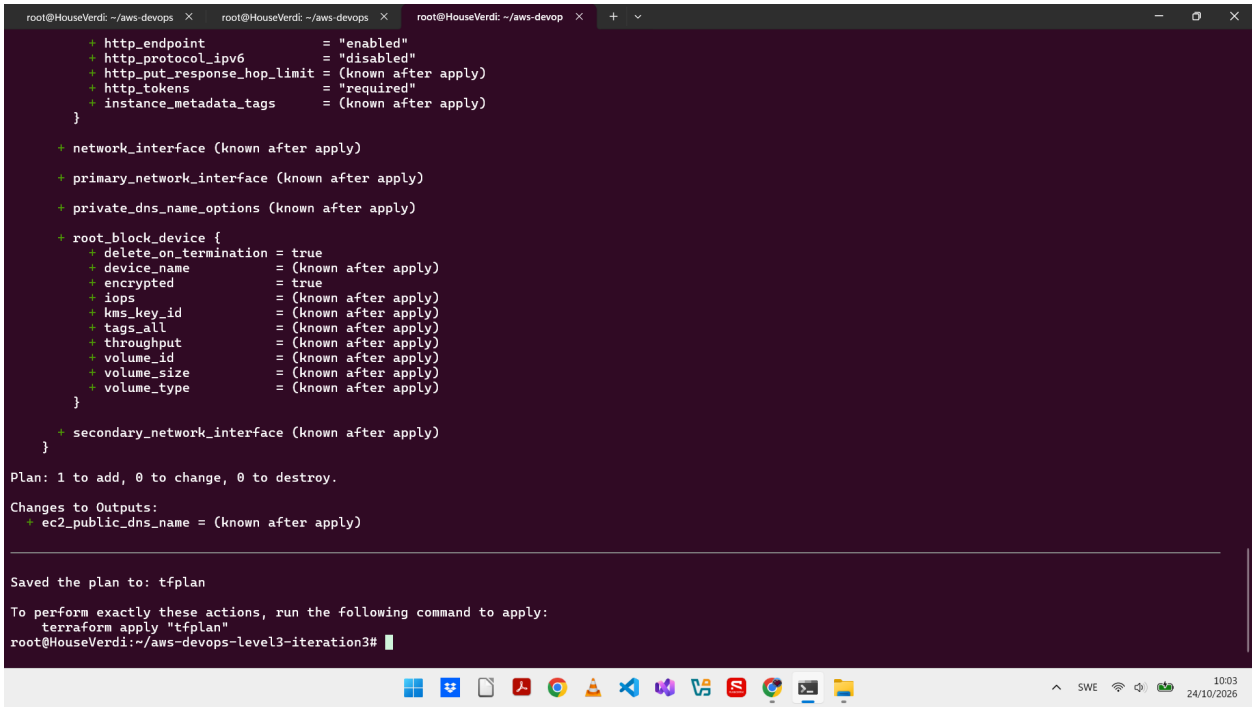


Figure 4. Saved plan for one new EC2 instance

Provisioning and cloud-init

terraform apply tfplan created the instance in 13 seconds. Its DNS name was ec2-13-62-54-224.eu-north-1.compute.amazonaws.com. I then checked cloud-init over SSH as ubuntu using the existing ~/.ssh/id_rsa_level3 key. The result was status: done.

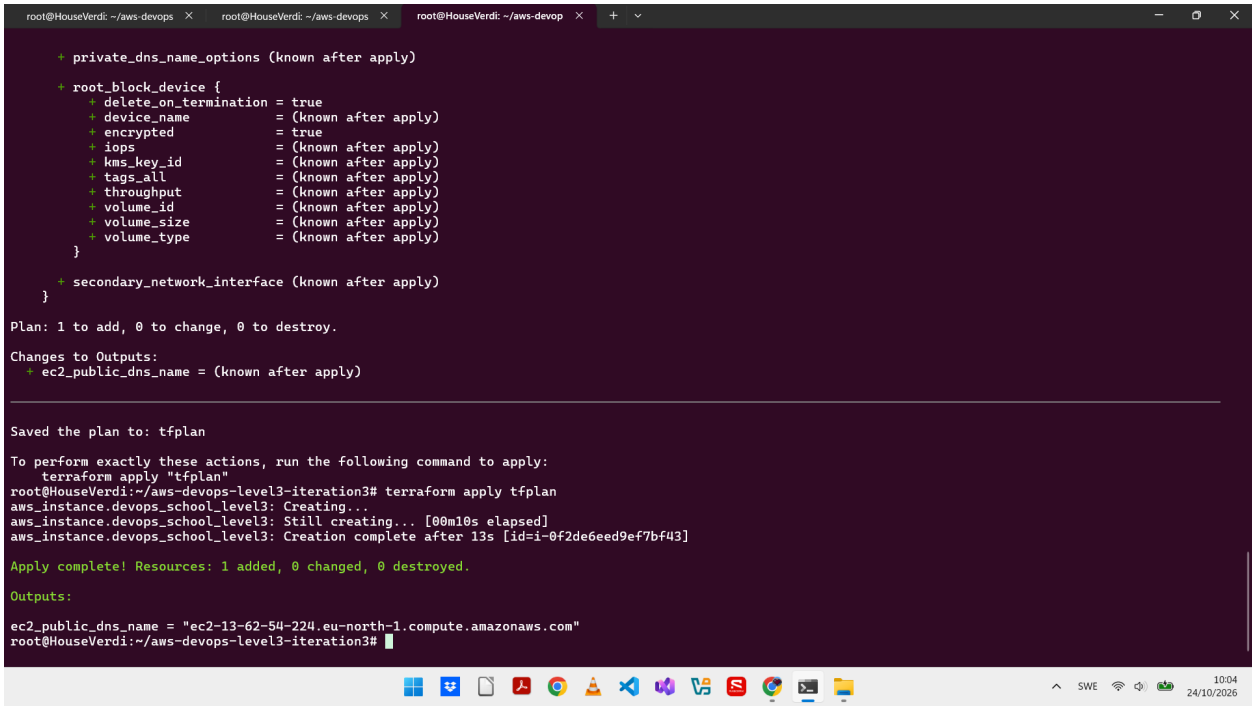


Figure 5. Instance created with ID and public DNS name

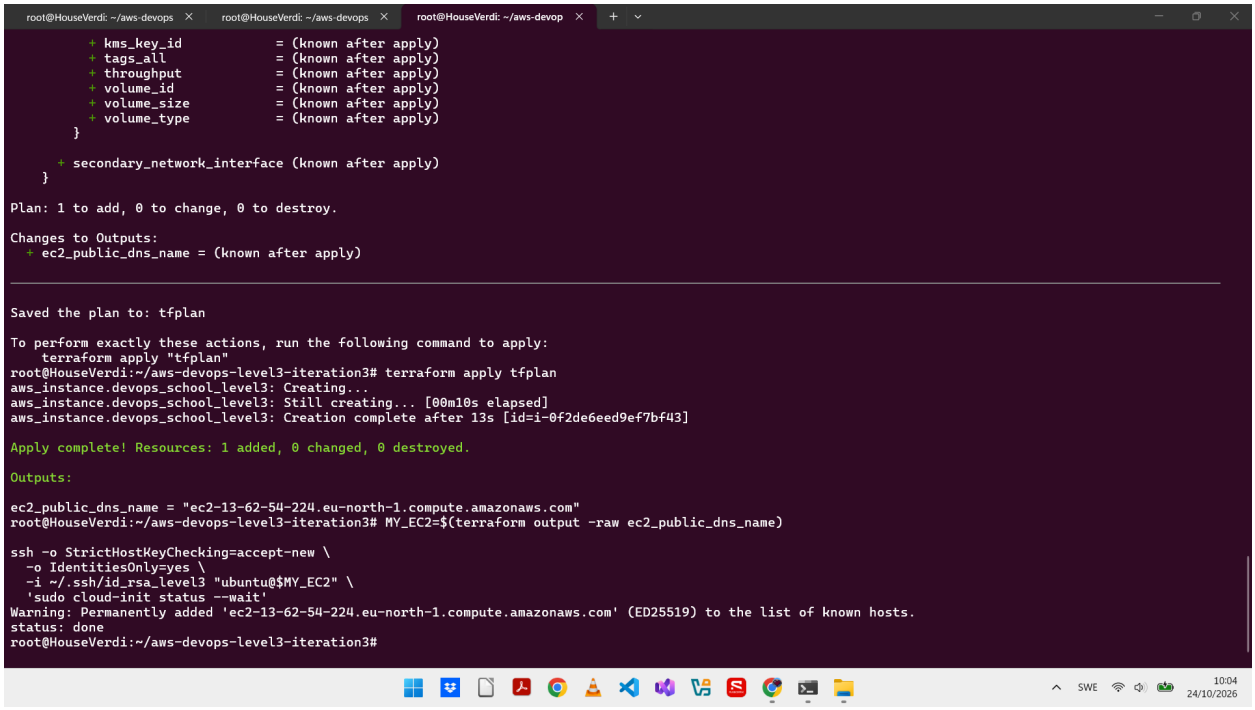


Figure 6. Cloud-init reports status done

Automatic installation and deployment

The log shows Nginx installation, git clone, install -m 644, a successful nginx -t, and service enablement and reload. Git was already present in the image and was checked by apt. No packages needed upgrading. Cloud-init finished after 41.05 seconds of uptime.

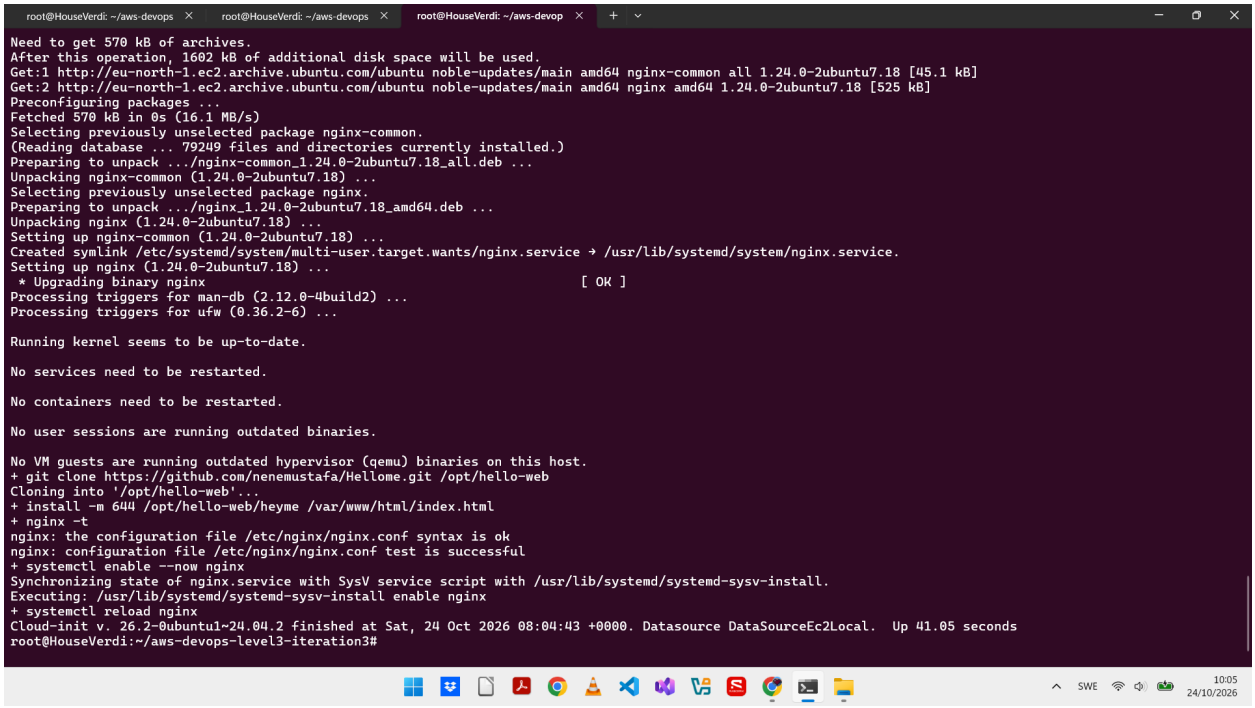


Figure 7. Cloud-init log with automatic deployment and successful Nginx test

Difference from iteration 2

In iteration 2, I deployed the website manually over SSH. In iteration 3, those steps also moved into user-data. Here, SSH was used to read status and logs. The website did not need to be copied manually after provisioning.

HTTP and automated content test

From the local WSL terminal, curl returned HTTP/1.1 200 OK, Content-Type text/html and Content-Length 499. The HTML contained the heading Hey me!. A separate content test using grep -qi returned YES and exit code 0. I enabled pipefail so that curl failures would also affect the test exit code.

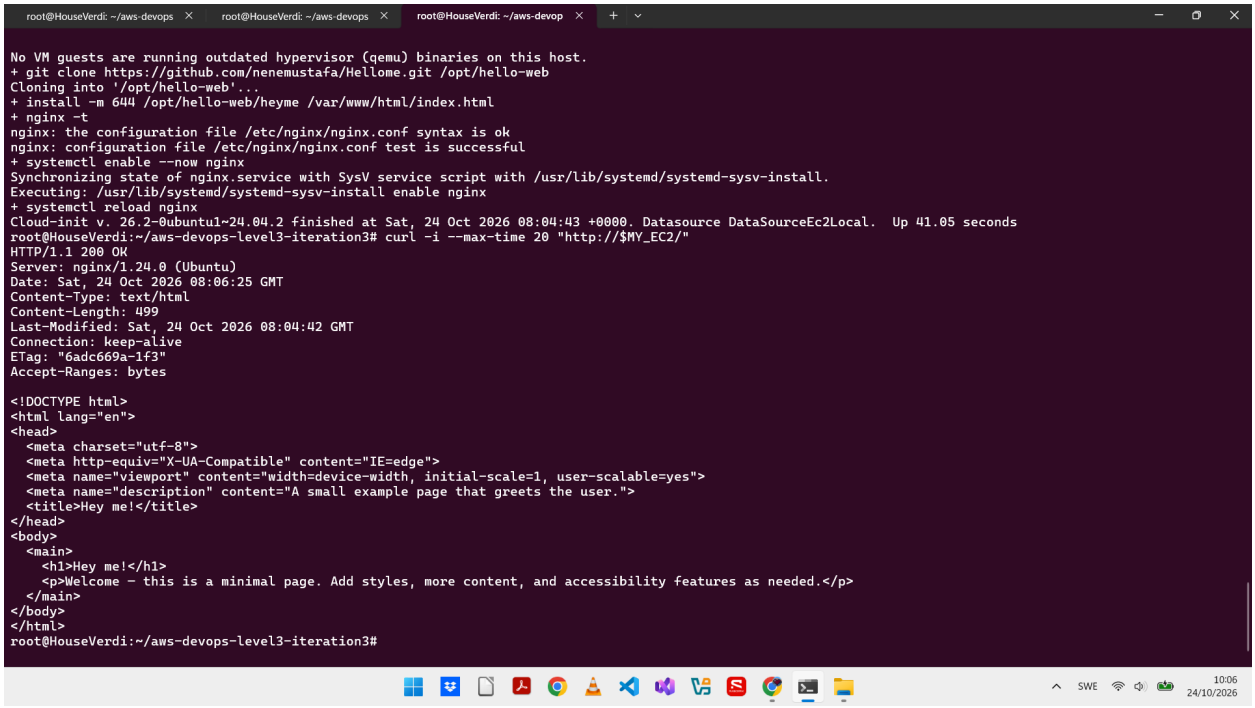


Figure 8. HTTP 200 and website HTML

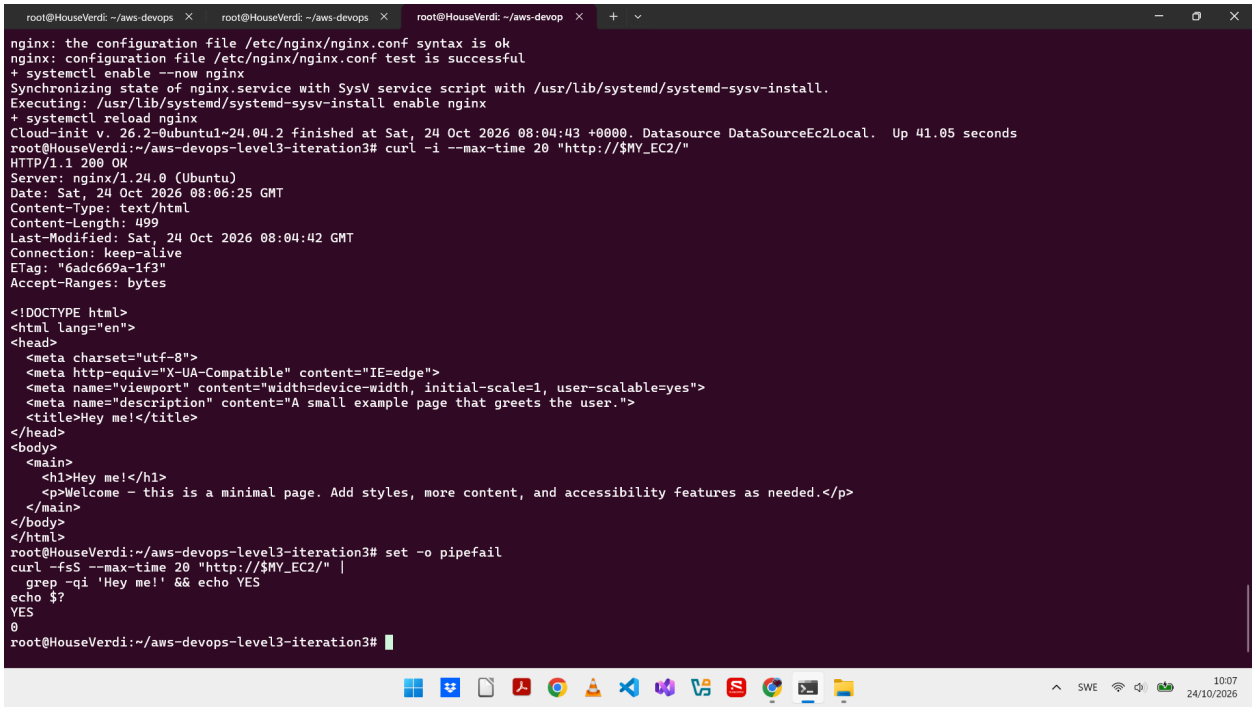


Figure 9. Content test with YES and exit code 0

Browser and Terraform verification

The browser displayed the expected website at 13.62.54.224. A new terraform plan returned No changes. This checks infrastructure against the Terraform configuration; website content was verified separately through the HTTP test and browser.

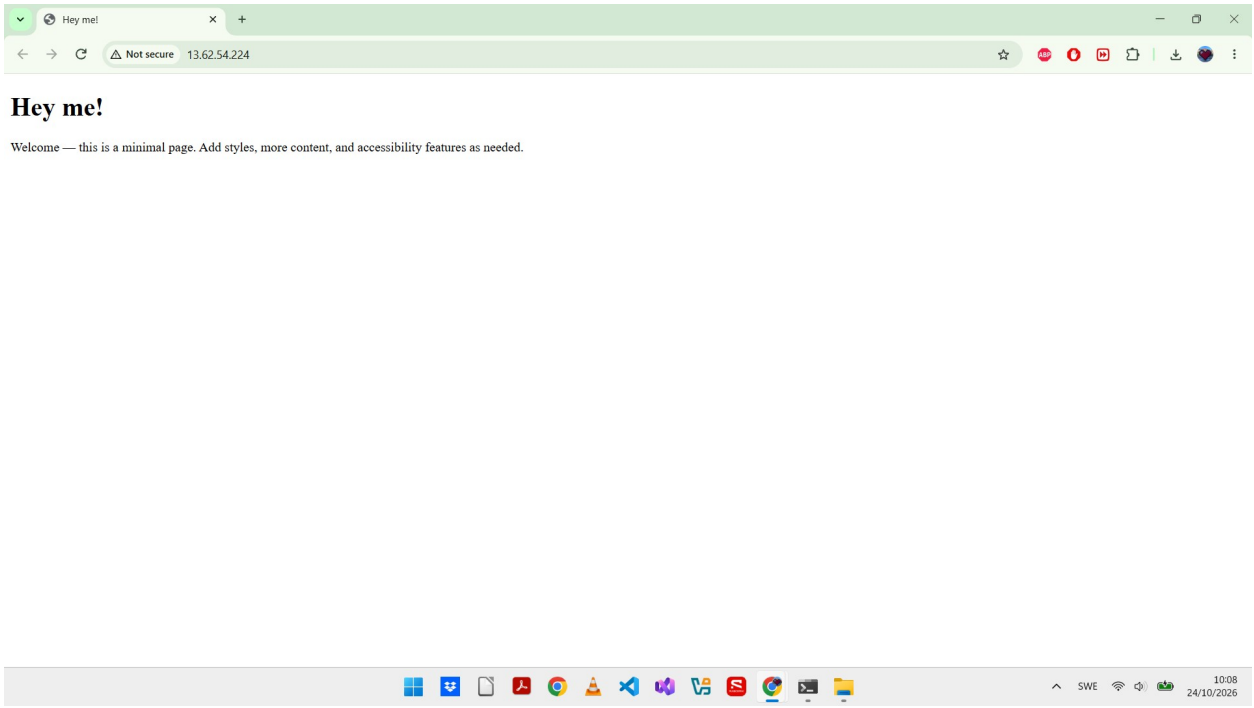


Figure 10. The Hey me website at the correct public IP

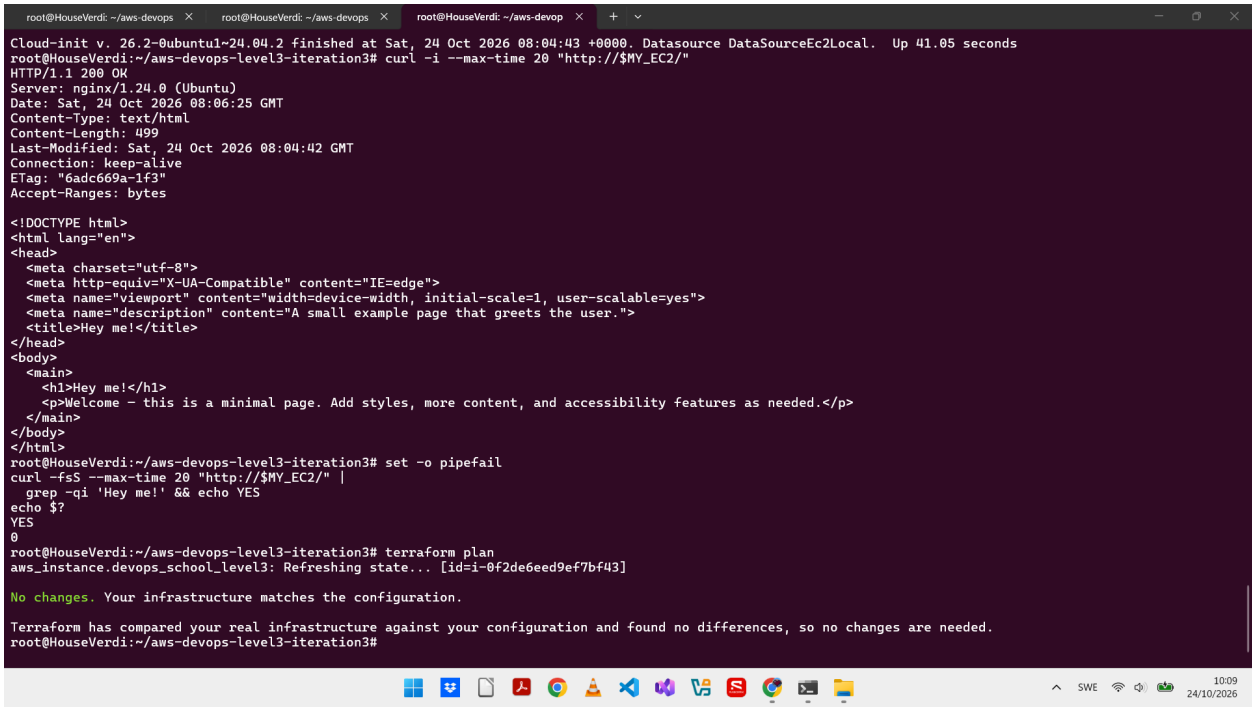


Figure 11. Terraform reports No changes

Cleanup and verified removal

After saving the configuration, terminal history and cloud-init log, I created destroyplan: 0 to add, 0 to change and 1 to destroy. terraform apply destroyplan removed the instance in 20 seconds. AWS CLI confirmed terminated on 24 October 2026 at 10:16 Swedish time.

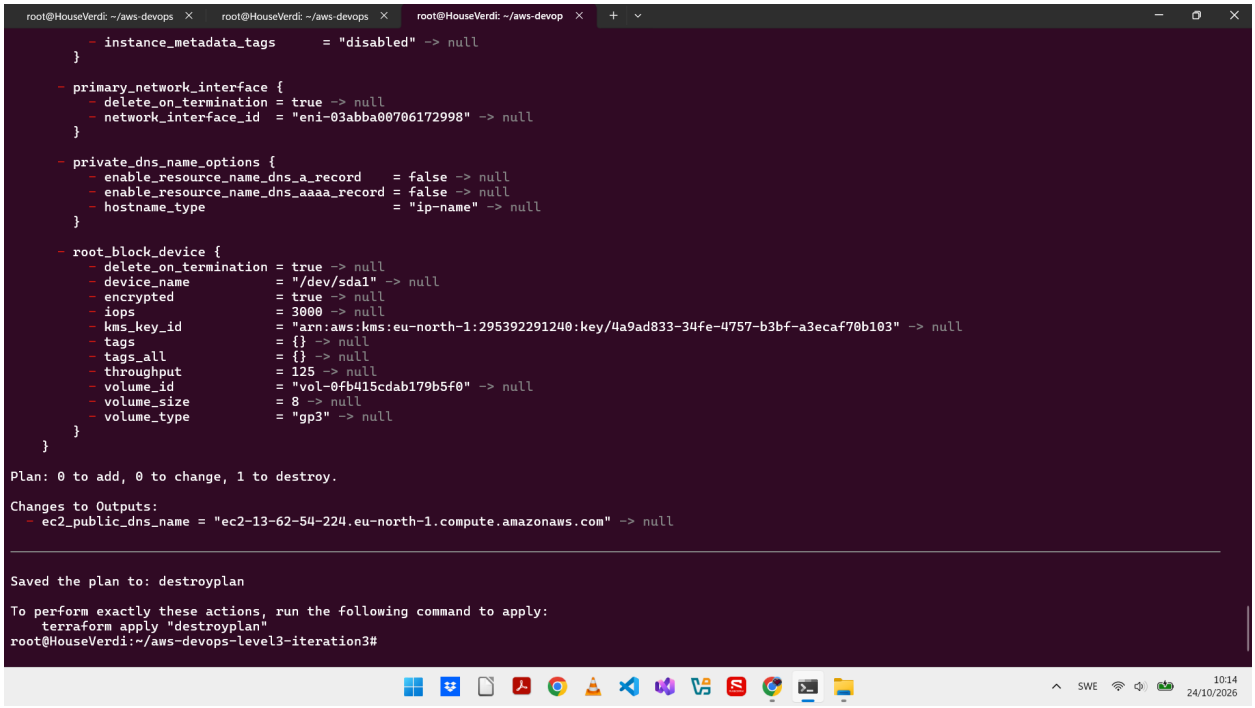


Figure 12. Removal plan with one instance and encrypted root disk

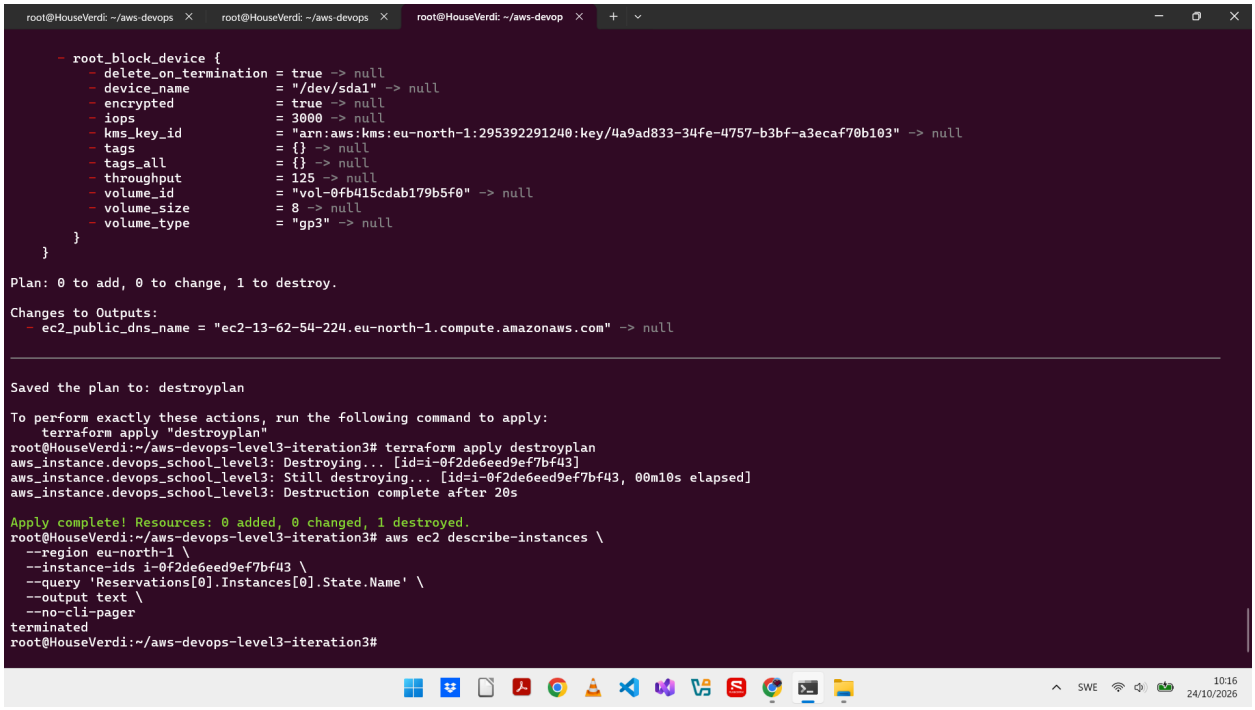


Figure 13. One resource destroyed and final state terminated

Adaptations and conclusion

Adaptations for Ubuntu

I replaced the guide's yum, service, ec2-user and /usr/share/nginx/html with apt-get, systemctl, ubuntu and /var/www/html. I used the website file heyme instead of the guide's hellome/html/* path. cloud-init runs the script as root, so sudo is unnecessary inside user-data.

Result and improvements

Terraform and user-data produced a working web server with automatic deployment. set -e stops the script on command failures, set -u detects undefined variables and set -x makes commands visible in the log. The content test uses case-insensitive matching because the page shows Hey me! whereas the guide searches for Hey Me!.

Possible improvements include HTTPS, pinning a specific Git commit and allowing the script to run again when /opt/hello-web already exists. The current git clone command is intended for a fresh instance. Nginx was enabled for automatic startup; no reboot test was performed.

The root disk had delete_on_termination=true. The key pair and security group existed outside Terraform and were not included in the removal. The IP and DNS in this report refer to tests before cleanup.

Evidence

Level3 Iteration3(3).sh; level3-3-konfiguration.txt; level3-3-terminal-history.txt; level3-3-cloud-init.log and embedded screenshots. The server log uses UTC: 08:04:43 corresponds to 10:04:43 Swedish time on 24 October.

Appendix Terraform base files

providers.tf

```
terraform {
  required_version = ">= 1.5.0"

  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 6.0"
    }
  }
}

provider "aws" {
  region = var.aws_region
}
```

variables.tf

```
variable "aws_region" {
  type = string
}

variable "ami_id" {
  type = string
}

variable "subnet_id" {
  type = string
}

variable "main_sg_id" {
  type = string
}

variable "ec2_type" {
  type = string
}

variable "ec2_name" {
  type = string
}

variable "keypair" {
  type = string
}
```

Appendix instance and variable values

main.tf

```
resource "aws_instance" "devops_school_level3" {
  ami                         = var.ami_id
  instance_type               = var.ec2_type
  subnet_id                   = var.subnet_id
  key_name                    = var.keypair
  vpc_security_group_ids      = [var.main_sg_id]
  associate_public_ip_address = true

  user_data                   = file("${path.module}/user-data.web.sh")
  user_data_replace_on_change = true

  metadata_options {
    http_tokens = "required"
  }

  root_block_device {
    encrypted = true
  }

  tags = {
    Name = var.ec2_name
  }
}
```

outputs.tf

```
output "ec2_public_dns_name" {
  description = "Public DNS name of the EC2 instance"
  value       = aws_instance.devops_school_level3.public_dns
}
```

terraform.tfvars

```
aws_region = "eu-north-1"
ami_id     = "ami-06fb1802e2b757f09"
subnet_id  = "subnet-05d4dca4820d2eb94"
main_sg_id = "sg-0fb87d0f08a213466"
ec2_type   = "t3.micro"
ec2_name   = "devops-school-level3.tabish-admin"
keypair    = "devops.school.level3.tabish-admin"
```

Appendix user-data and verification commands

user-data.web.sh

```
#!/bin/bash
set -euxo pipefail
export DEBIAN_FRONTEND=noninteractive

apt-get update
apt-get upgrade -y
apt-get install -y nginx git

git clone https://github.com/nenemustafa/Hellome.git /opt/hello-web
install -m 644 /opt/hello-web/heyme /var/www/html/index.html

nginx -t
systemctl enable --now nginx
systemctl reload nginx
```

Verification from local terminal

```
MY_EC2=$(terraform output -raw ec2_public_dns_name)
ssh -o IdentitiesOnly=yes -i ~/.ssh/id_rsa_level3 \
  "ubuntu@$MY_EC2" 'sudo cloud-init status --wait'
ssh -o IdentitiesOnly=yes -i ~/.ssh/id_rsa_level3 \
  "ubuntu@$MY_EC2" \
  'sudo tail -n 60 /var/log/cloud-init-output.log'
curl -i --max-time 20 "http://$MY_EC2/"
set -o pipefail
curl -fsS --max-time 20 "http://$MY_EC2/" |
  grep -qi 'Hey me!' && echo YES
echo $?
terraform plan
```

Removal and final check

```
terraform plan -destroy -out=destroyplan
terraform apply destroyplan
aws ec2 describe-instances --region eu-north-1 \
  --instance-ids i-0f2de6eed9ef7bf43 \
  --query 'Reservations[0].Instances[0].State.Name' \
  --output text --no-cli-pager
```